Privacy-sanitized historical report

Characterizing Local Breadth in Language Generation
Fan Wei and Jon Kleinberg

This is a public-candidate copy, not the historical original.
The unchanged historical original remains in the private archive.

Two personal filesystem locations were previously removed from the report.
Private source-repository locators have now been replaced by local routes
into the current self-contained reproducibility package. These local routes
are navigation aids; they do not identify the historical immutable artifacts.

Mathematical statements, conclusions, qualifications, original source commit
identities and original page numbering are unchanged. Older source-version
counts remain historical. See reproducibility/CERTIFICATE.md for current scope.

The portable mathematical source has comment-only privacy changes. Its
public source hashes differ; its non-comment mathematical content is checked.

No claim is made that Lean verifies every manuscript prose proof sentence.
This notice is an additional unnumbered cover, replacing the earlier notice.

Formal verification of *Characterizing Local Breadth in Language Generation*

Consolidated editor-facing verification report - November 2, 2026

Manuscript: Jon Kleinberg and Fan Wei, *Characterizing Local Breadth in Language Generation*

Authoritative text: `Nov 1.tex` (November 1, 2026)

Manuscript SHA-256: `eee1524496f215c15f2278bd39c6f7b24214d0e1e1dd783a552cca3233977249`

Verification reference: `Nov2-Exact-Statements-v1`

Access: The source and audit repositories linked below are private; access must be granted to reviewers.

Executive summary - Level K certification

Overall conclusion. The November 2 Lean 4 verification package and its independent AI correspondence audit classify **all 65 numbered mathematical assertions in the November 1 manuscript at Level K** under the stated formal model: **18 theorems, 30 lemmas, 8 claims, 7 corollaries, and 2 propositions**. There are **no remaining Q, P, or K* classifications among these 65 assertions**. Theorem 5.8 restates Theorem 2.6, but is counted as a separate numbered occurrence here; counting that mathematical result only once gives 64 distinct numbered results, as in the independently supplied comparison report. **All five remarks are excluded** from the certification; the 14 numbered definitions are classified as definitions, not as proved theorems.

Definition of Level K (the project's standard, not a built-in Lean certification level). A numbered paper assertion is designated **K** only when all four requirements are met:

- **K1 - Faithful statement.** An independent correspondence review compares the paper and the *actual elaborated Lean statement* clause by clause, including hypotheses, order of quantifiers, definitions, dimension/threshold ranges, and conclusion.

- **K2 - Kernel-checked proof and axioms.** The statement has a completed proof accepted by Lean's kernel, with no `sorry`, `admit`, or new nonstandard axiom; transitive dependencies use only the standard axioms `propext`, `Classical.choice`, and `Quot.sound`.

- **K3 - Formal representation bridges.** Where a mathematical object is encoded differently (for example, trees, copies, topology, limits, the earlier paper's language family, or the game-state domain), substantive equivalence or the needed implication is itself supported by kernel-checked Lean results rather than an unproved verbal identification.

- **K4 - Full theorem strength.** No conclusion is weakened and no *additional unproved* premise is slipped into the paper-facing theorem; the quantifier order and the documented standing model are preserved. If an ambiguity requires a particular mathematical reading, that choice is disclosed rather than silently assumed.

Prior-work dependency conclusion. **None of the 65 mapped Lean proofs uses an unformalized theorem from an earlier paper as an additional axiom or unproved premise.** Dependencies on the preceding 1D development and the concrete previously published zero-family construction are present as **Lean definitions and proofs**; other cited literature provides mathematical context, not unchecked proof obligations. The pinned **Mathlib** dependency is itself formalized mathematics, not an unverified manuscript result. The right-hand dependency column in Section 5 makes this status explicit for every row. This does **not** claim that all background literature, informal prose, or figures have been separately formalized.

Evidence. The recorded checks passed for all 62 one-dimensional and 55 higher-dimensional proof-bearing modules; a transitive axiom sweep covers **5,271** project declarations, and complete project replay rechecks **3,874 theorems** against the pinned Mathlib base. A clean *project* build passed using cached pinned external dependencies. The optional complete-from-empty Mathlib replay timed out and is **not** reported as successful.

Scope and human-review caveats. Level K is the conclusion of a **separate AI mathematical correspondence audit**, not human mathematical sign-off, and does not certify the line-by-line correctness of every written proof. **Corollary 1.6 is K without an empty-family exception:** the [pinned one-dimensional source](reproducibility/lean/BanachDensity1D/Oct31Bridges.lean) contains the explicit theorem `BanachDensity1D.Oct31.finite_point_rank_all`, which handles both nonempty and empty countable families. Its type, standard-axiom dependencies, and successful build are recorded in the [Corollary 1.6 follow-up evidence](reproducibility/evidence/VERIFICATION-RESULTS.md). The original 65-row audit remains a historical snapshot; this documentation states the proved, already-pinned wrapper explicitly. Exact findings and audit records are linked below.

Where readers can reproduce the work:

- **Start here - reproduction instructions:** [Open verified resource](reproducibility/README.md)
- **Immutable complete source checkout:** [Open verified resource](reproducibility/lean/)
- **Versioned audit package and checksum manifests:** [Open verified resource](reproducibility/evidence/VERIFICATION-RESULTS.md)
- **65-item theorem-by-theorem technical report:** [Open verified resource](reproducibility/evidence/ASSERTION-CERTIFICATION.md)
- **Independent mathematical completeness audit:** [Open verified resource](reproducibility/evidence/ASSERTION-CERTIFICATION.md)
- **Exact elaborated Lean types and direct proof dependencies:** [Open verified resource](reproducibility/evidence/ELABORATED-REGISTRY.json)
- **Separate accessible reading certificate:** CERTIFICATE.md

Contents

- [Executive summary](#executive-summary--level-k-certification)
- [Purpose, scope and Level K](#0-purpose-scope-and-what-a-level-k-result-means)
- [Verification layers and version history](#1-how-far-to-trust-the-result)
- [Formal model and assumptions](#2-what-the-formalization-assumes-and-represents)
- [Main mathematical conclusions](#3-main-formalized-mathematical-conclusions)
- [Representation bridges and completed gaps](#4-substantive-representation-bridges-and-earlier-limitations-that-were-discharged)
- [All 65 numbered results](#5-complete-numbered-result-coverage)

- [Notices and scope exclusions](#6-source-issues-limitations-and-scope-exclusions)

- [Proof-route differences](#7-differences-between-the-written-proof-and-the-lean-development)

- [Reproducibility instructions](#8-reproduce-the-formalization-and-verify-the-evidence)

- [Reading order](#9-reading-order-for-editors-and-referees)

---

0. Purpose, scope, and what a Level K result means

The purpose of this document is to let an editor or mathematical referee understand **what was formalized, what was checked, how to reproduce the checks, and where the formal verification does not apply**. Unlike the technical audit log, this is a reader-facing account of the delivered result. The companion `CERTIFICATE.md` places selected paper results beside their actual Lean theorem types and supplies a complete item index. The exact-machine-readable data remain authoritative when a brief presentation in this report omits low-level detail.

The four-part **K1-K4 definition** is stated in the executive summary and applies throughout the coverage tables below. A matching theorem name or a successful build alone does not earn K. Previous labels **K***, **P**, and **Q** denoted qualified correspondence, a stronger-than-paper premise, and a substantive missing bridge, respectively; the completed project's independent AI review reports none as remaining among the included numbered assertions. The formerly noted Corollary 1.6 boundary is closed by the already-pinned theorem `BanachDensity1D.Oct31.finite_point_rank_all`, which includes the empty family.

The reported 65-item scope is exactly: 18 theorems, 30 lemmas, 8 claims, 7 corollaries, and 2 propositions. The active manuscript scan also finds 14 numbered definitions, five remarks, two unnumbered theorem summaries and one guiding question. Definitions are represented in the development and examined through the statements that use them; they are not additional proved theorems. The remarks, figures, introductory prose and verbatim proof routes are not separately certified as Level K statements.

The formalization verifies **mathematical truth of the statements**, sometimes via arguments organized differently from the printed proofs. It does not imply that every sentence of the manuscript has been independently proved in Lean, or that the author approved each Lean statement after reading its elaborated type. It is appropriate to say the results were **AI-audited**, not **human-certified**.

1. How far to trust the result

Verification layer	What it establishes	Evidence and precise limitation
Lean build	Every requested proof module is elaborated and accepted by Lean.	62 proof-bearing 1D modules and 55 HD modules, including two outside the principal aggregate; logs in `exact-statement-verification/checks/`.
Transitive axiom sweep	For the imported project constants, no nonstandard proof axioms occur.	5,271 project-origin declarations across 126 modules. Only `propext`, `Classical.choice`, `Quot.sound` are reported.
Project kernel replay	Complete project-origin logical declarations replay against a Mathlib-only base with no project constants.	Successful: 5,248 logical constants, 3,874 theorems. 23 compiler-only unsafe-recursion artifacts are excluded and individually logged.
Clean project reproduction	Source compiles in a separately prepared project worktree with a new project build directory.	Passed. External pinned dependency caches were reused; this is not a rebuild of all Mathlib proofs from empty storage.
Frozen-source preservation	Historical declarations and proof files were preserved.	54 approved 1D and 97 approved HD declarations, zero drift; no dependency-pin or manuscript change.
Independent mathematical correspondence	A separate AI reader compared numbered paper assertions, exact Lean types, definitions, and substantive bridges.	Final audit reports 65 K, 0 Q/P. This is not an independent kernel implementation or human mathematical sign-off.

| **Written-proof review** | The AI reviewer examined substantial written proof arguments, and identified editorial/prose caveats. | Does not certify every line of the manuscript; this is distinct from statement verification. |

The full project replay uses the installed Lean 4 kernel implementation and a pinned imported Mathlib environment. An **optional** attempt to replay *all imported Mathlib proofs from an empty kernel environment* reached a 900-second time limit; **that optional check is not claimed to have passed**. Earlier 180-second and aggregate-budget timeouts were preserved, while their separate longer reruns passed. It would be incorrect to count the optional timeout as a failed project proof or as a successful full-Mathlib replay.

1.1 Fixed version and provenance

Item	Fixed identity
Manuscript SHA-256	`eee1524496f215c15f2278bd39c6f7b24214d0e1e1dd783a552cca3233977249`
HD base before exact-statement additions	`33488acdbaa066db3532b562790b1b6d4011bb76`
Additive mathematical source commit	`d892cc89de751604c3593f2e5f60da8e7429cfc3`
Published complete source/evidence HEAD	`dff2c2b99dc5c685164e37dc70b0818336096fe7`
Imported 1D dependency pin	`efe7a25711657a5cfd0dad8d902392ce4d711e92`
Published audit package commit	`f09ab4e8791d82a5232b45634762c49905022515`
Lean	`leanprover/lean4:v4.35.0`
Pinned Mathlib	`56a005855ea17e0b0958f73ec801ae140b0a1679`

The additive HD changes were recorded as 18 new Lean source files (1,829 lines) without modifying existing mathematical declarations. The actual 1D import is the pinned commit shown above; a separate 1D publication branch has mathematically identical tracked source but must not be substituted silently for the pin.

2. What the formalization assumes and represents

2.1 Languages, enumerations, generators and validity

A language is a subset of the integer lattice, and the family is modeled as a **nonempty countable family of infinite languages**. A full enumeration is an infinite sequence whose range is exactly its target language; repetitions may occur. Generators are deterministic causal maps from finite observed input lists to output strings. The paper's **global freshness requirement** is enforced: an output is neither among the inputs received so far nor among earlier outputs. Eventual validity requires that after some target/enumeration-dependent time all outputs belong to the true language. No computational time bound or probability distribution is imposed.

The 1D development uses `Set Z`; the HD development uses `Set (Pt d)` with `Pt d = Fin d -> Z`. A theorem stated for `d >= 2` is not silently given a `d = 1` interpretation. The Section 7 threshold is `h(V)=u(V)log(2+V)` with **positive nondecreasing** `u`, *not* an unmentioned assumption `u>=1`.

2.2 Density and nondegenerate rectangles

The relative lower Banach-density numerator is `|A intersect B intersect R|`, divided by `|B intersect R|`. The filtered version retains the **infimum over finite axis-parallel rectangles** satisfying `m_B(R) >= s h(V(R))`; only the *outer monotone limit* is represented as a supremum over natural-number filter levels. Empty eligible collections receive density 1, as the manuscript specifies. No replacement by a supremum over rectangles is made. In the threshold-normalization bridges, scaling `h` by a positive factor preserves the resulting density, and full protected-tree certificates are transported where the theorem requires them.

2.3 Histories, trees, copies and finite games

The 1D structural value uses refinement histories, pairing scores and admissible well-founded obstruction trees. The HD characterization uses **protected trees with actual occurrence structure**, finite stopping games, partial topological decorations, and a **single global assignment** tested against countable admissible vertex obstructions. In Section 7, accuracy copies `(K,k)` carry the literal dyadic quota `(alpha(K)-2^(-k))_+`; repeated occurrences are preserved and the broad child rule includes **equal-language** edges. The final representation bridges prove equivalence of the required *certificate-existence propositions*, not a false one-to-one isomorphism of all possible tree encodings.

2.4 Formal state domain in Appendix A

The historical `RegionTree.PotentialProps` demanded inequalities throughout the closure of arbitrary legal Black/White claims and arbitrary activations. That condition could be stronger than the manuscript's intended conditional. The exact-statement extension provides `OnPlay.Properties`: quantitative requirements apply to the strategy's actual pre-, post-Black and post-activation states, and the necessary locality comparisons extend to the required hypothetical probes. Lemma A.2 is proved from these **localized premises**. Proposition A.5 then uses `PlayScaleConditions` and the on-play lemma, **without** invoking Claim A.6 as a shortcut to the conditional. A separate proved implication shows the concrete broad-domain potentials of Claim A.6 satisfy the localized properties. This closes the reported P qualification under the explicitly recorded interpretation, without claiming equivalence of the two different premise domains.

3. Main formalized mathematical conclusions

3.1 One-dimensional exact characterization

Theorem 1.5 identifies `b_n^(comb)(K)` and `b_n^(top)(K)` and proves both optimality directions: every universally eventually valid admissible generator has an adverse enumeration at each target, while a **single** generator achieves the structural guarantee for **all** targets and their enumerations. Theorem 1.8 gives the corresponding exact structural characterization, and Theorems 2.5, 4.1, 5.1 and 5.23 formalize the constructive and adversarial components. Theorem 2.6 and the Section 5 results supply the history-topological interpretation and the forest invariants. Corollaries 1.6 and 3.3 concern finite Cantor--Bendixson **point rank**, without a common rank bound.

3.2 Higher-dimensional trade-offs and exact achievable profiles

Theorems 6.1 and 6.2 prove the separation of individual optima from simultaneously achievable profiles, including the explicit `1/16` gap and the `1/4`, `1/2`, and `x+y<=1/2` quantities. Theorem 7.1 establishes the fixed-quota protected-tree criterion in both directions under positive nondecreasing `u`. Theorem 7.2 gives the exact achievable-profile criterion using **dyadic accuracy copies**; Theorem 7.7 and Corollaries 7.8--7.9 give topological and obstruction-tree formulations. Lemmas 7.10--7.17 verify the scan/capture mechanism, sewing, simulator and **specific constructed generator** needed for the sufficient direction.

3.3 Geometric finite-rank lower bounds and endpoint obstruction

Lemma 8.1 proves a *single* strategy for all eligible rectangles in a **finite packet**, including the printed `d>=1` range. Theorem 8.2 proves simultaneous near-half guarantees for all finite point-rank targets with no uniform bound on their ranks. Theorems 7.3 and 9.1 and Lemma 9.2 establish the nonattainability of the exact one-half endpoint in higher dimensions under the stated threshold conditions.

3.4 Appendix constructions and exact frontiers

Appendix A verifies the potential-based conditional machinery and the explicit construction of scales and potentials. Appendix B proves the zero structural value for the **previous paper's actual printed family**, the full range `[0,1/2]` in one dimension, and the realizability of **every product region** `?_i[0,alpha_i]` as an exact simultaneous profile frontier. The two staged-family claims yield both the structural lower and upper bounds for the same constructed languages, despite a duplicated lower sign in the printed heading of Claim B.5.

4. Substantive representation bridges and earlier limitations that were discharged

Former concern	New checked source	What the new result adds
Positive `u` for the fixed-quota theorem	`ExactStatements/PositiveThreshold.lean`	Both Theorem 7.1 directions and complete protected-tree cutoff transport, not just scalar density normalization.
Reciprocal versus dyadic accuracy	`ExactStatements/CopySchedule.lean`	Certificate-existence equivalence preserving complete child coverage and original paths.
Literal copied occurrences	`ExactStatements/CopyOccurrences.lean` and `PaperExports.lean`	Manuscript-facing occurrence-tree equivalence and exact profile export.
Nonstrict copy forest	`ExactStatements/CopyForest.lean`, `CopyEnumeration.lean`	Actual adjacent-copy ancestry, specified copy capture, and filtered density for the constructed generator.
Reverse vertex obstruction presentation	`ExactStatements/VertexRealization.lean`, `VertexCharacterization.lean`	Countable vertex presentations with terminal paths and one global assignment.
Commitment boundary	`ExactStatements/Commitments.lean`	Includes the response to the final committing input.
Sewing under positive `u`	`ExactStatements/SewingPositive.lean`	Both printed log and weighted estimates with the printed dimensional coefficient.
Abstract Appendix A conditional	`ExactStatements/OnPlayPotential.lean`, `OnPlayRectangle.lean`	A.2/A.5 conditional proofs using precisely specified on-play hypotheses and necessary probes.
Previous zero-family identity	`BanachDensity1D/PriorZeroValue/Correspondence.lean`	Equality to the printed family, not merely a surrogate zero-value example.
Staged-family upper bound	`ExactStatements/ProfileInternal.lean`	Actual upper and equality theorems complementing the B.4 lower bound.

These additions are mathematically meaningful **new Lean proofs** relative to the previous Lean library. They are *not necessarily new mathematical proofs relative to the paper*: some follow or reorganize the manuscript arguments; some use alternative formal routes.

5. Complete numbered-result coverage

The tables below cover **each of the 65 numbered proved assertions**. **Status K** means K1-K4 as defined above, under the project's documented standing model. Corollary 1.6 is supported by the separately identified empty-family wrapper in Section 6.2. **"No"** in the **Unformalized prior theorem required?** column means that the Lean proof does not assume a theorem from an earlier paper without a Lean proof. Formalized lemmas from **Mathlib**, the pinned 1D library, and the HD project *are allowed*: these are checked formal dependencies, not excluded prior results. The extra note on Proposition B.1 explains why its cited earlier construction is not an unformalized premise.

The last column identifies mathematical content or an interpretative boundary; it is not an unrecorded downgrade. **Exact Lean statements, binder order, source hashes, and direct/transitive dependencies** are available in the [elaborated-type register](reproducibility/evidence/ELABORATED-REGISTRY.json) and [full technical report](reproducibility/evidence/ASSERTION-CERTIFICATION.md).

Introduction and Part I (Sections 1--5)

Paper	Level	Formal counterpart	Unformalized prior theorem required?	Correspondence note
Theorem 1.5	**K**	`BanachDensity1D.thm_intro_one_dimensional`	**No**	Equality of structural values, adversarial upper bound, and one simultaneous lower generator.
Corollary 1.6	**K**	`BanachDensity1D.Oct31.finite_point_rank_all`	**No**	Finite point rank, no common rank bound, including the empty-family case.
Theorem 1.7	**K**	`BanachDensityHD.thm_intro_higher_nonsimultaneous` / `BanachDensityHD.thm_main_not_simul`	**No**	Same witness family and generators for the ordinary density separation and trade-off.
Theorem 1.8	**K**	`BanachDensity1D.thm_main_oneD`	**No**	One exact lower generator and a target-wise adversarial upper enumeration.
Claim 2.2	**K**	`BanachDensity1D.claim_cn_eventual_interval_bound`	**No**	Eventual relative-interval bound for a fixed refinement chain.
Claim 2.4	**K**	`BanachDensity1D.claim_bn_tree_path_bound`	**No**	Tree-path witness at a strict threshold below b_n.
Theorem 2.5	**K**	`BanachDensity1D.thm_lower`	**No**	Exact achievability, eventual validity, and global freshness.
Theorem 2.6	**K**	`BanachDensity1D.thm_topological_bn`	**No**	Topological-combinatorial equality for b_n.
Lemma 3.2	**K**	`BanachDensity1D.lem_finite_rank_witness`	**No**	Finite-rank witness with a strict rank decrease.
Corollary 3.3	**K**	`BanachDensity1D.cor_finite_rank_b`	**No**	Explicit finite-n rank loss and limiting half-density.
Theorem 4.1	**K**	`BanachDensity1D.thm_upper`	**No**	Upper bound against every admissible eventually valid generator.
Lemma 4.2	**K**	`BanachDensity1D.lem_upper_race`	**No**	Finite adversarial race on previously unused points.
Theorem 5.1	**K**	`BanachDensity1D.thm_fixed_epsilon`	**No**	One epsilon-dependent generator works for every target.
Lemma 5.4	**K**	`BanachDensityHD.HistoriesAux.lem_history_topology_gen`	**No**	Basic-neighborhood/topological construction preserving refinement history.
Lemma 5.5	**K**	`BanachDensity1D.lem_finest_history_topology`	**No**	Finest compatible topology using countable diagonal arguments.
Lemma 5.6	**K**	`BanachDensityHD.HistoriesAux.lem_obstruction_closure_gen`	**No**	Closure characterized by arbitrary countable well-founded branching.

| Corollary 5.7 | **K** | `BanachDensity1D.cor_safety_interior` | **No** | Safety and unsafety as interior/closure properties. |

| Theorem 5.8 | **K** | `BanachDensity1D.thm_topological_bn_restated` | **No** | Topological b_n formula restated in proof context. Numbered restatement of Theorem 2.6; counted separately. |

| Theorem 5.9 | **K** | `BanachDensity1D.thm_topological_main` | **No** | Full generation theorem in terms of topological b. |

| Corollary 5.10 | **K** | `BanachDensity1D.cor_protected_set` | **No** | Basic finite-protection and safety equivalence. |

| Lemma 5.12 | **K** | `BanachDensity1D.lem_boards` | **No** | Finite-trace board existence and exact size bounds. |

| Lemma 5.13 | **K** | `BanachDensity1D.lem_forest_invariant` | **No** | Protected-path and ancestry invariants of the forest. |

| Lemma 5.14 | **K** | `BanachDensity1D.lem_uniform_board` | **No** | Board quota max(0,s-1/N), including freezing. |

| Lemma 5.15 | **K** | `BanachDensity1D.lem_memory` | **No** | Conditional memory/record-language inclusion. |

| Lemma 5.16 | **K** | `BanachDensityHD.ForestAux.capture_gen` | **No** | Finite-evidence eventual capture, without target identification. |

| Lemma 5.17 | **K** | `BanachDensity1D.lem_validity` | **No** | Only finitely many invalid outputs. |

| Lemma 5.18 | **K** | `BanachDensityHD.lem_pairing_game` | **No** | Total rounded-half bound for the filtered pairing game. |

| Corollary 5.19 | **K** | `BanachDensity1D.cor_regular_board_density` | **No** | Regular-board density guarantee. |

| Lemma 5.20 | **K** | `BanachDensity1D.lem_copies_chain` | **No** | Adjacent accuracy copies of a language form an ancestor chain. |

| Lemma 5.21 | **K** | `BanachDensity1D.lem_capture_fixed_copy` | **No** | Eventual capture of each specified copy. |

| Lemma 5.22 | **K** | `BanachDensity1D.lem_fixed_copy_density` | **No** | One fixed-copy quota approaches the optimal value. |

| Theorem 5.23 | **K** | `BanachDensity1D.thm_oneD_lower_conclusion` | **No** | One generator achieves b(K) on all targets and enumerations. |

Higher dimensions (Sections 6--9)

| Paper | Level | Formal counterpart | Unformalized prior theorem required? | Correspondence note |

|:--|:--:|:--|:--:|:--|

| Theorem 6.1 | **K** | `BanachDensityHD.Oct31.separation_gap_one_sixteenth` | **No** | Explicit universal 1/16 separation, with witnesses before h. |

| Theorem 6.2 | **K** | `BanachDensityHD.thm_main_not_simul` / `BanachDensityHD.family7_witnesses` | **No** | Exact optima 1/4 and 1/2, simultaneous other-target guarantee, trade-off. |

| Lemma 6.3 | **K** | `BanachDensityHD.lem_validity_commitment` | **No** | Finite commitment under eventual validity. |

| Lemma 6.4 | **K** | `BanachDensityHD.ExactStatements.distinct_input_block_forcing` | **No** | Block forcing from a distinct initial input list. |

| Lemma 6.5 | **K** | `BanachDensityHD.prop_tradeoff` | **No** | Trade-off from horizontal/vertical layer races. |

| Claim 6.6 | **K** | `BanachDensityHD.claim_horizontal_test` | **No** | Adversarial horizontal test with a complete finite strip prefix. |

| Theorem 7.1 | **K** | `BanachDensityHD.ExactStatements.fixed_quota_suff_positive` / `BanachDensityHD.ExactStatements.fixed_quota_nec_positive` | **No** | Both protected-tree directions for positive nondecreasing u. |

| Theorem 7.2 | **K** | `BanachDensityHD.ExactStatements.exact_profile_occurrences_positive` | **No** | Exact achievable profiles using dyadic copy occurrence trees. |

| Theorem 7.3 | **K** | `BanachDensityHD.Oct31.finite_rank_near_half_all` / `BanachDensityHD.end_main` | **No** | Near-half at every finite point-rank target and strict endpoint obstruction. |

| Theorem 7.7 | **K** | `BanachDensityHD.TreeCorrespondence.paper_topological_characterization` | **No** | Occurrence-tree/topological neighborhood equivalence. |

| Corollary 7.8 | **K** | `BanachDensityHD.TreeCorrespondence.exact_vertex_obstruction_characterization` | **No** | Global assignment then arbitrary countable vertex obstructions. |

| Corollary 7.9 | **K** | `BanachDensityHD.TreeCorrespondence.PaperDecoration.finite_protection` | **No** | Finite protection on good basic neighborhoods. |

| Lemma 7.10 | **K** | `BanachDensityHD.lem_hd_capture` | **No** | Critical-vertex capture for the actual HD forest. |

| Lemma 7.11 | **K** | `BanachDensityHD.ExactStatements.sewing_positive` | **No** | Exact weighted sewing bound for positive nondecreasing u. |

| Lemma 7.12 | **K** | `BanachDensityHD.lem_hd_help` | **No** | Simulation/virtual play preserves real outputs. |

| Lemma 7.13 | **K** | `BanachDensityHD.ExactStatements.commitments_including_final_reply` | **No** | Commitment includes the reply to the last committing input. |

| Lemma 7.15 | **K** | `BanachDensityHD.ExactStatements.adjacent_copy_ancestry` | **No** | Nonstrict scan preserves adjacent equal-language accuracy copies. |

| Lemma 7.16 | **K** | `BanachDensityHD.ExactStatements.capture_specified_copy` | **No** | Literal nonstrict copy-forest capture. |

| Lemma 7.17 | **K** | `BanachDensityHD.ExactStatements.dyadic_fixed_copy_density_positive` | **No** | Fixed-copy density from the actual dyadic forest and generator. |

| Lemma 8.1 | **K** | `BanachDensityHD.Nov1.local_rectangle_strategy_positive_dimension` | **No** | One game strategy for every rectangle in a finite packet; d>=1. |

| Theorem 8.2 | **K** | `BanachDensityHD.Oct31.finite_rank_near_half_all` | **No** | Near-half simultaneously without a uniform bound on finite ranks. |

| Theorem 9.1 | **K** | `BanachDensityHD.end_main` | **No** | Exact-half endpoint impossibility for d>=2. |

| Lemma 9.2 | **K** | `BanachDensityHD.end_strip` | **No** | Adversarial strip game including off-strip White responses. |

Appendix A: potential method

| Paper | Level | Formal counterpart | Unformalized prior theorem required? | Correspondence note |

|:--|:--:|:--|:--:|:--|

| Lemma A.2 | **K** | `BanachDensityHD.ExactStatements.OnPlay.quotas_of_properties` | **No** | Local potential strategy under on-play hypotheses and necessary probes. |

| Claim A.3 | **K** | `BanachDensityHD.claim_discardsmall` | **No** | Discarded small remainders have a controlled total fraction. |

| Claim A.4 | **K** | `BanachDensityHD.claim_quotassuffice` | **No** | Quotas on local pieces imply the desired global rectangle guarantee. |

| Proposition A.5 | **K** | `BanachDensityHD.ExactStatements.conditional_rectangle_on_play` | **No** | True conditional implication under G1-G3, without invoking A.6. |

| Claim A.6 | **K** | `BanachDensityHD.claim_cf_potential_existence_uniform` | **No** | Uniform concrete scale and potential construction. |

Appendix B: constructions

| Paper | Level | Formal counterpart | Unformalized prior theorem required? | Correspondence note |

|:--|:--:|:--|:--:|:--|

| Proposition B.1 | **K** | `BanachDensity1D.PriorZero.prop_prior_zero_value` | **No** (prior family reconstructed in Lean) | Exact printed earlier sumset family has b_n(K0)=0 for every n. |

| Lemma B.2 | **K** | `BanachDensity1D.cor_full_range` | **No** (B.1/1D construction formalized) | Every exact optimum alpha in [0,1/2] occurs. |

| Lemma B.3 | **K** | `BanachDensityHD.prop_profile_realization` | **No** (construction formalized here) | Every prescribed product profile region occurs for an appropriate family. |

| Claim B.4 | **K** | `BanachDensity1D.ProfileAux.Staged.bK_lower` | **No** | Lower structural value at each rational cutoff of staged family. |

| Claim B.5 | **K** | `BanachDensityHD.ExactStatements.staged_bK_upper` / `BanachDensityHD.ExactStatements.staged_bK_exact` | **No** | Literal repeated heading and intended upper bound; resulting equality. |

5.1 Selected statement-correspondence sign-off sheets

The following selected sign-offs highlight *actual hypotheses and quantifier order*, where a superficial name-to-name mapping would have been insufficient. These are explanatory extracts of the final independent AI review. They are not a replacement for inspection of the elaborated types linked above.

Theorem 1.5: one universal generator, not one per target

Printed mathematical content. For every target, the topological and combinatorial values agree. Every competing valid generator has an adverse full enumeration at each target, but there exists **one** lower-bound generator that works for *every* target and *every* full enumeration. The latter existence quantifier must come before the target and enumeration quantifiers.

Lean counterpart. `BanachDensity1D.thm_intro_one_dimensional` is a conjunction of three parts. The second begins `? G`, followed by `? K in X`, then `? w`; the third begins `? G` and only then quantifies `? K in X, ? w`. The construction supplies global availability and eventual validity. The equality `bn = bnTop` is in the first conjunct. This retains the central nonidentification/simultaneous-achievability distinction.

Reading boundary. Global freshness is a standing property of admissible generators. Repeated inputs in a full enumeration remain allowed. This is a statement-level check; it does not claim that every intermediate 1D forest proof step is written in the same order as the manuscript.

Theorems 7.1 and 7.2: positivity, complete trees, and literal copies

Printed mathematical content. Section 7 assumes `h(V)=u(V) log(2+V)` for positive nondecreasing `u` and `d>=2`. The fixed-quota criterion has a sufficiency direction and a necessity direction with *every positive additive slack*. The exact-profile theorem uses dyadic accuracy copies `(K,k)`, where the tree must contain all compatible children, including same-language copies at other indices.

Lean counterparts. `fixed_quota_suff_positive` and `fixed_quota_nec_positive` prove both directions for `0<u`, using `protected_normalization`. The new `DyadicCopyProtected` predicate uses `max(alpha(L)-2^(-k),0)`, and `all_dyadic_iff_reciprocal` proves the required certificate-existence equivalence; it is not based only on cofinal scalar numerical limits. `exact_profile_occurrences_positive` then exports the final theorem using original copy occurrence trees.

Reading boundary. Proving a profile with an arbitrary *different* forest would not establish Lemmas 7.15-7.17. The new Lean uses `scanParent false`, exact copy ancestry and capture, and the actual `cfgOf` simulator and sewing conclusion. The source/registry records the `d>=2` and `u>0` assumptions, and the first output within the committing prefix is separately handled by Lemma 7.13's new wrapper.

Corollary 7.8: one global assignment before every obstruction

Printed mathematical content. A single assignment of geometric thresholds and radii must be good for **every** admissible countable obstruction presentation in the specified sense. The theorem cannot be reduced to "each obstruction admits its own convenient assignment."

Lean counterpart. `exact_vertex_obstruction_characterization` places the existential `? s a` before the universal quantifiers over the countable vertex type and its obstruction tree. `all_vertex_obstructions_iff`, with the concrete `Address` realization, proves the correspondence between actual vertex presentations and language-path representations, without a hidden finite-branching hypothesis.

Reading boundary. The proved bridge is an equivalence of the terminal-path properties needed for the theorem. It does not assert an isomorphism of all occurrence structures after merging duplicate decorations.

Lemma A.2 and Proposition A.5: the conditional actually proved

Printed mathematical content. Lemma A.2 assumes locality and (P1)-(P4) for the game-state evaluations required by the prescribed strategy and its hypothetical one-move comparisons. Proposition A.5 says that whenever scales and local potentials satisfy (G1)-(G3), one White strategy protects every rectangle in a finite packet. An explicit construction of *one* potential family in Claim A.6 is not an adequate substitute for proving A.5's general conditional.

Lean counterparts. `OnPlay.Properties` uses `PlayState` for actual pre- and intermediate states and `ProbeState` for the locality comparisons. `OnPlay.quotas_of_properties` verifies the invariant and local quotas. `PlayScaleConditions` encodes the uniform scale/threshold antecedent, and `conditional_rectangle_on_play` derives the appropriate dyadic tree and the packet conclusion from **those assumed conditions**. The separate `of_bookkeeping` and `playScaleConditions_of_scaleConditions` provide only the valid broad-to-local implication.

Reading boundary. Arbitrary activations are part of the *old stronger* bookkeeping-domain interpretation, not automatically required by the manuscript's prescribed strategy. The exact development does not assume or claim a false equivalence of the two state domains. The independent audit's final K determination is explicitly **under this precise on-play reading**.

Proposition B.1 and Claims B.4-B.5: the constructed families themselves

Printed mathematical content. Proposition B.1 uses the family from a previous manuscript, not simply an example with zero value. The Appendix B profile construction uses the staged rational-cut languages; Claim B.4 establishes their lower structural bound and B.5's **written proof** establishes the corresponding upper bound, although the B.5 heading repeats a lower sign.

Lean counterparts. `PreviousPaperIntervals`, `printedLanguage_eq`, and `printedFamily_eq` bridge the earlier paper's actual interval/offset construction to the internal zero-family object. `PriorZero.prop_prior_zero_value` then proves the claimed zero values for that family. For the staged family, `Staged.bK_lower`, `staged_bK_upper`, and `staged_bK_exact` establish the lower, upper and resulting equality on the **same languages**.

Reading boundary. The family-decoding check adopts the coordinate convention supported by the earlier primary source and preserves the exceptional offset term by equality. The stale previous-paper theorem number is an editorial citation issue, not a Lean axiom. B.5's duplicated heading should be fixed in the manuscript even though the two mathematical inequalities are available as formal theorems.

Lemma B.3: an exact product frontier, not one convenient profile

Printed mathematical content. For any admissible threshold and any prescribed sequence in `[0,1/2]`, the constructed countable family has **exactly** the product region of simultaneously achievable density profiles; in particular, its coordinatewise greatest profile is achieved by a common generator.

Lean counterpart. `BanachDensityHD.prop_profile_realization` contains an injectively indexed family, an explicit `? G` before index and enumeration quantification, an equivalence for every indexed profile, and an equivalence for every full family profile. These are separate conjuncts in the exported type. The internal staged-family result supplies a matching structural upper bound rather than deriving "exact" from achievability alone.

Reading boundary. The family may depend on the threshold as allowed by the statement. The full proof of the filtered upper bound uses the actual witness construction; it does not assume the paper's adversarial sketch as an external theorem.

5.2 Numbered definitions: representation index

The **14 numbered definitions** are identified separately below. They are **not failures of Level K**: a definition is not a theorem needing a proof. The status **Defined in Lean** indicates an encoded mathematical object; **Background definition** denotes a previously standard concept used for context rather than a new proof obligation. The last column distinguishes mathematical background from any required *unproved external theorem*. The reader should consult the [full elaborated-type register](reproducibility/evidence/ELABORATED-REGISTRY.json
) for definitions that
are inlined into the corresponding theorem type.

Paper definition	Status	Mathematical object	Unformalized prior theorem required?	Source representation
Definition 1.2	**Background definition**	Lower asymptotic density	**No** (not a theorem)	Background definition; 1D notation and finite prefixes
Definition 1.3	**Defined in Lean**	Lower Banach density	**No**	`BanachDensityHD.banach`, target-relative bridge to `deltaH`
Definition 1.4	**Defined in Lean**	Filtered lower Banach density	**No**	`BanachDensityHD.testInf`, `deltaH`, `deltaH_tendsto`
Definition 2.1	**Defined in Lean**	Relative intervals, raw pairing score	**No**	`BanachDensity1D.def_raw_pairing_score`, layer and interval definitions
Definition 2.3	**Defined in Lean**	Admissible obstruction trees	**No**	`BanachDensity1D.LTree.Admissible`, `TreeCorrespondence.PaperObstruction`
Definition 3.1	**Defined in Lean**	Cantor--Bendixson point rank	**No**	`BanachDensity1D.CBrank`, derivative constructions
Definition 5.2	**Defined in Lean**	Safe and unsafe histories	**No**	`BanachDensity1D.def_safe_unsafe_history`
Definition 5.3	**Defined in Lean**	History topology	**No**	`BanachDensityHD.historyTopology`; 1D history topology
Definition 5.11	**Defined in Lean**	Protected sets	**No**	`BanachDensity1D.def_history_protected_set`
Definition 7.4	**Defined in Lean**	Geometric feasibility	**No**	`BanachDensityHD.GeomFeasible`; finite `StopGame.Feasible`
Definition 7.5	**Defined in Lean**	Protected trees	**No**	`BanachDensityHD.IsProtectedTree`; occurrence-tree equivalence
Definition 7.6	**Defined in Lean**	Geometric decorations	**No**	`BanachDensityHD.IsQGoodDecoration`; paper decoration
Definition 7.14	**Defined in Lean**	Protected trees on accuracy copies	**No**	`BanachDensityHD.ExactStatements.DyadicCopyProtected`

| Definition A.1 | **Defined in Lean** | Activation rule | **No** | `BanachDensityHD.RegionTree.activationStep` |

Interpretation of cited prior work. Proposition B.1 states a consequence for a family defined in the authors' earlier paper, but the earlier family itself is reconstructed and connected to its printed representation by `printedLanguage_eq` and `printedFamily_eq`, and the required zero-value conclusion is proved in `BanachDensity1D.PriorZero`. Thus **B.1 is not accepted as an unformalized literature axiom**. The remaining references to older papers (including historical motivation for language generation and previous density results) are **background references**, not extra logical premises of the 65 Lean proofs. The full transitive axiom sweep is what supports the absence of a hidden nonstandard axiom; the AI statement audit separately addresses stronger-than-paper hypotheses.

6. Source issues, limitations and scope exclusions

6.1 No general claim about the five remarks

The five remarks are excluded from the required K inventory. This is a **scope decision**, not a finding that those remarks are false. In particular, Remark 5 begins with **one interval `J`**. Language layers within that interval are disjoint. An example with two *overlapping test windows* may challenge a broader arbitrary-packet scalar sufficiency interpretation but does not refute the literal single-interval rounded-half calculation. The independent review explicitly withdrew its earlier unqualified criticism. The report does not claim formal verification of Remark 5's entire prose.

6.2 Corollary 1.6: empty-family case formalized

The printed corollary allows any countable family of infinite languages, including the empty family. The original mapped result `BanachDensity1D.cor_intro_one_dimensional_rank` assumes `LangClass X` (which includes nonemptiness), but **the pinned 1D library already contains** [`BanachDensity1D.Oct31.finite_point_rank_all`](reproducibility/lean/BanachDensity1D/Oct31Bridges.lean), whose only family
assumptions are countability, infinitude of each member, and pointwise finite Cantor-Bendixson rank. Its Lean proof performs an explicit case split. For a nonempty family it invokes `cor_intro_one_dimensional_rank`; for an empty family it constructs the existing globally admissible fresh generator and discharges the vacuous target-dependent clauses. The [recorded theorem type, axiom check, and build
evidence](reproducibility/evidence/VERIFICATION-RESULTS.md) report success with only
standard axioms. **Thus this numbered assertion has an exact K counterpart with no added nonemptiness premise.** This was a documentation/mapping correction, not a new mathematical proof.

6.3 Editorial points distinguished from mathematical proof obligations

- **Claim B.5:** The printed claim heading repeats `b(L_q) >= alpha_q` from B.4, while its proof develops `b(L_q) <= alpha_q`. Both the literal lower result and the intended upper/equality are checked in the staged-family library; the heading remains an editorial correction to consider.

- **Previous-paper reference:** A `[KWBanach, Theorem 2.4]` citation does not precisely identify the printed zero-family construction in the cited version. The actual Lean correspondence follows the source construction and proves `printedLanguage_eq` and `printedFamily_eq`; the citation is an editorial issue.

- **Absent appendix pointer:** A prose reference to an arbitrary-threshold sewing appendix has no corresponding appendix in the authoritative manuscript. The claimed appendix extension is not treated as an additional numbered result.

- **Finite-value/strict-threshold prose and finite-precision embedding:** The independent review records supporting interpretation details and some omitted expository justifications, not an identified counterexample to a numbered statement. Their written derivations are not silently incorporated into the 65 K theorem types.

The complete wording and provenance of these findings are in [Mathematical notices](reproducibility/CERTIFICATE.md) and [Independent completeness audit](reproducibility/evidence/ASSERTION-CERTIFICATION.md).

7. Differences between the written proof and the Lean development

A Lean proof need not duplicate the order or every intermediate step of the paper to certify its conclusion. The most consequential documented route choices are:

1. **Histories and trees.** Lean codes some trees by families of finite paths and some by original vertex occurrences. Proved representation bridges are used where the printed theorem quantifies over actual occurrences; a naive label-quotient equivalence is not assumed.

2. **Theorem 7.1.** Earlier Lean proofs covered `u>=1`; the new proof normalizes `u>0`, transports cutoff labels throughout the protected tree and then transports achievable density.

3. **Theorem 7.2.** A reciprocal schedule can represent the same achievable region, but the paper's dyadic certificate is recovered via an explicit tree-restriction/relabelling proof.

4. **Lemmas 7.15--7.17.** The exact copy construction uses nonstrict parent scans and same-language copy ancestry; the `cfgOf` output set and actual simulator are used, not an unrelated witness generator.

5. **Corollary 7.8.** Abstract histories and countable vertex obstruction presentations are connected by a proved terminal-path bridge; no finite-branching simplification is substituted.

6. **Lemma 7.13.** A new wrapper includes the final commitment input's response, addressing the timing boundary of the old formalization.

7. **Lemma 8.1.** The finite game is a simultaneous **packet** requirement; the formal proof does not treat independently feasible rectangle scores as automatically feasible together.

8. **Lemma A.2 and Proposition A.5.** Locality is specified on actual game states plus needed hypothetical probes. The conditional implication is proved on these premises rather than inferred only from the concrete potentials of A.6.

9. **Proposition B.1.** The zero-value theorem is applied to the actual reconstructed printed family with coordinate and exceptional-term equalities.

10. **Claim B.5.** The Lean file exposes the intended upper structural bound and equality for the actual staged family despite the repeated printed lower heading.

No assertion here is that **all** differences between the manuscript's written proofs and Lean's proofs have been exhaustively catalogued. Goal K is statement-oriented; a line-by-line audit of every printed proof is a separate goal.

8. Reproduce the formalization and verify the evidence

8.1 Repository locations

Reproduction guide (recommended starting point): [Open verified resource](reproducibility/README.md)

Immutable source: [Open verified resource](reproducibility/lean/)

Full audit package: [Open verified resource](reproducibility/evidence/VERIFICATION-RESULTS.md)

Machine-readable exact statement inventory: [Open verified resource](reproducibility/manifests/THEOREM-MAP.json)

Elaborated Lean types / dependencies: [Open verified resource](reproducibility/evidence/ELABORATED-REGISTRY.json)

Raw verification summary: [Open verified resource](reproducibility/evidence/VERIFICATION-RESULTS.md)

Checksum manifest: [Open verified resource](reproducibility/manifests/SOURCE-SHA256SUMS)

The repositories are **private**: a referee will need an authorized GitHub account or a separately shared immutable bundle. Links are pinned to exact commits, rather than mutable branch tips. The audit package copies new source/evidence, but the complete historical proof base must be fetched from the corresponding immutable source commit and its pinned 1D dependency; the audit directory by itself is *not* a self-contained Lean checkout.

8.2 Reproduction commands

Install Git and `elan` (the Lean version manager). After making the two authorized repositories and their dependencies available as specified in the reproduction guide, use the following commands in the **HD checkout**:

lake build BanachDensityHD

lake build BanachDensityHD.ExactStatements

lake env lean exact-statement-verification/CompleteRegistry.lean

lake env lean exact-statement-verification/AllModuleAxioms.lean

lake env lean exact-statement-verification/ReplayOwnLibraries.lean

python3 exact-statement-verification/final_source_checks.py

At the **pinned 1D checkout** the recorded build is:

lake build MyFormalization BanachDensity1D BanachDensity1D.Nov1

The project was tested with process-local environment values `PYTHONDONTWRITEBYTECODE=1`, `TMPDIR=[temporary directory]`, and `GIT_OPTIONAL_LOCKS=0`. The HD Lake configuration has a **local-path Git dependency pointing to `[historical local 1D path; redacted]`**. An independent reproducer must install an authenticated clone at that path or use a **process-local Git URL mapping** as the guide explains. The published dependency definition and commit pin were not silently modified. No claim of effortless public one-command reproduction is made.

From the root of the **versioned audit directory**, verify the package and internal manifest (macOS syntax):

shasum -a256 -c exact-statement-verification/SHA256SUMS

shasum -a256 -c PACKAGE-SHA256SUMS

Builds, kernel replay, axiom sweeps and preservation checks have exact command arrays, work directories, exit codes, timestamps, toolchain versions and raw output logs in the `exact-statement-verification/checks/` directory. The former standalone reader report is preserved as historical evidence; corrected independent judgments are versioned rather than silently overwriting that history.

8.3 What reproducing those commands does *not* do

It does not rebuild all Mathlib declarations from an empty kernel or independently verify every prose statement. The complete project replay checks project declarations against a pinned Mathlib base and the standard Lean kernel. The optional whole-Mathlib-from-empty attempt timed out and remains disclosed. Running the commands also does not constitute a mathematician's manual approval of every paper-to-Lean correspondence.

9. Reading order for editors and referees

1. Read the **executive finding and scope qualifications** here.

2. Consult the companion CERTIFICATE.md for a **plain-English Lean glossary** and paper-versus-Lean signatures of the principal results.

3. Open the [65-row exact-statement correspondence](reproducibility/evidence/ASSERTION-CERTIFICATION.md) for each theorem, including every claim and Appendix B.

4. Open [REPRODUCE.md](reproducibility/README.md) for commands, [the audit package](reproducibility/evidence/VERIFICATION-RESULTS.md) for source/checksum records, and [the complete type register](reproducibility/evidence/ELABORATED-REGISTRY.json) when verifying a nuanced hypothesis or quantifier.

5. For concerns about an informal remark or an editorial typo, read the [independent audit's limitations](reproducibility/evidence/ASSERTION-CERTIFICATION.md) rather than inferring that the theorem proved something it does not say.

Conclusion. The certified mathematical development is substantive and reproducible: the current project's final logs report all 65 numbered assertions verified under its documented model, with standard-only axioms and successful complete project replay. The precise distinction between *kernel-checked formal proofs*, *AI-assessed manuscript correspondence*, *written-proof differences*, and *excluded remarks* is part of the certification-not an exception to be concealed.